Minicraft

Alyssa Hee  James Jiang

ECE241 Final Presentation

Inspiration: The OG Minicraft

https://github.com/alyssaHee/fpga-minicraft

Project Description

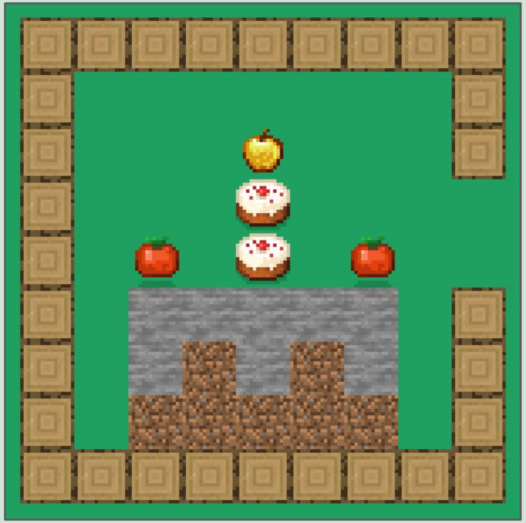
Objective

- Player's goal is to bake a cake in the wood box (create the image shown) by collecting and placing blocks

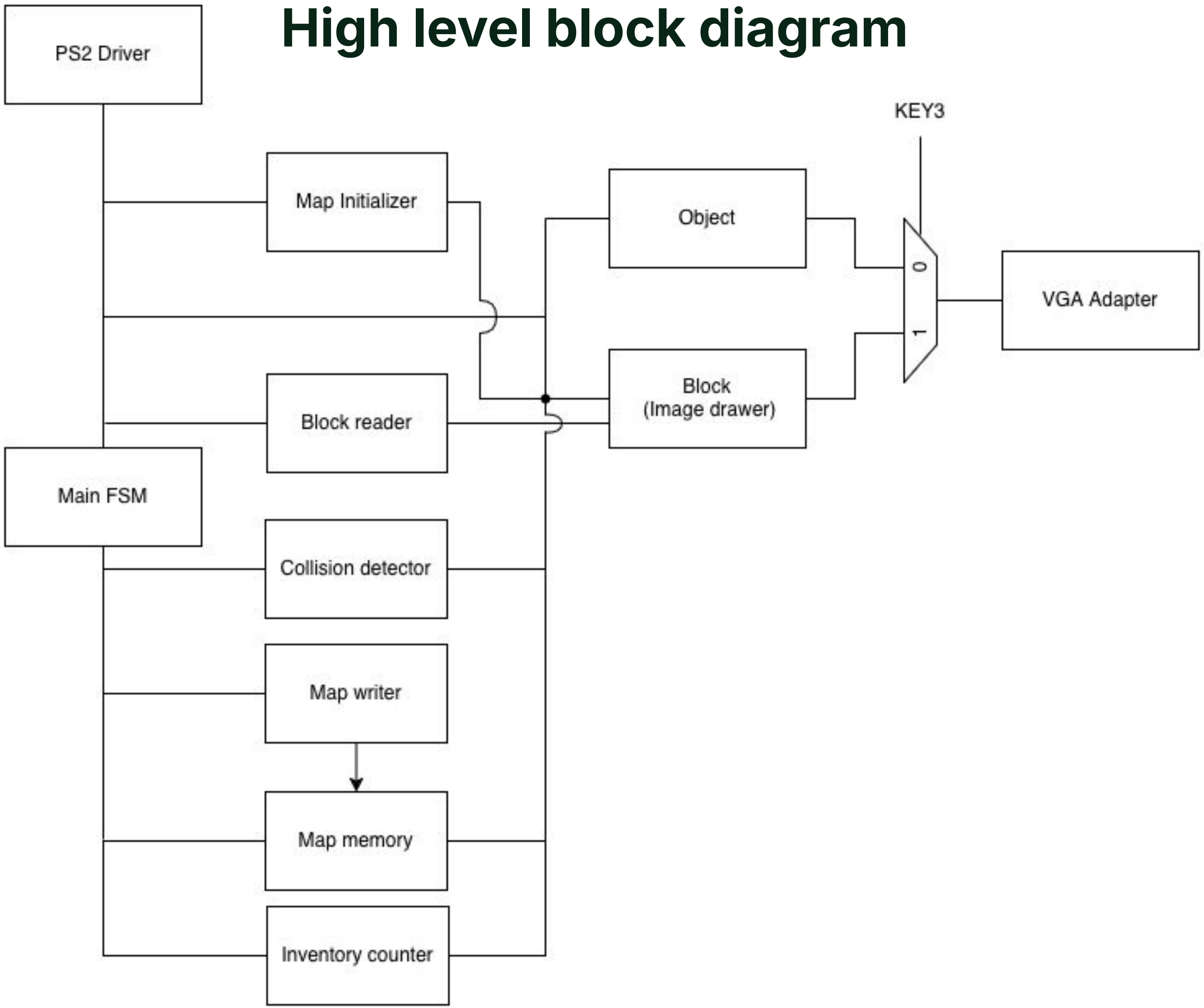
Controls

- `WASD` to move
- `KEY[0]` for breaking
- `SW[6:1]` for placing

- Inventory shown on `HEX[5:0]`



High level block diagram



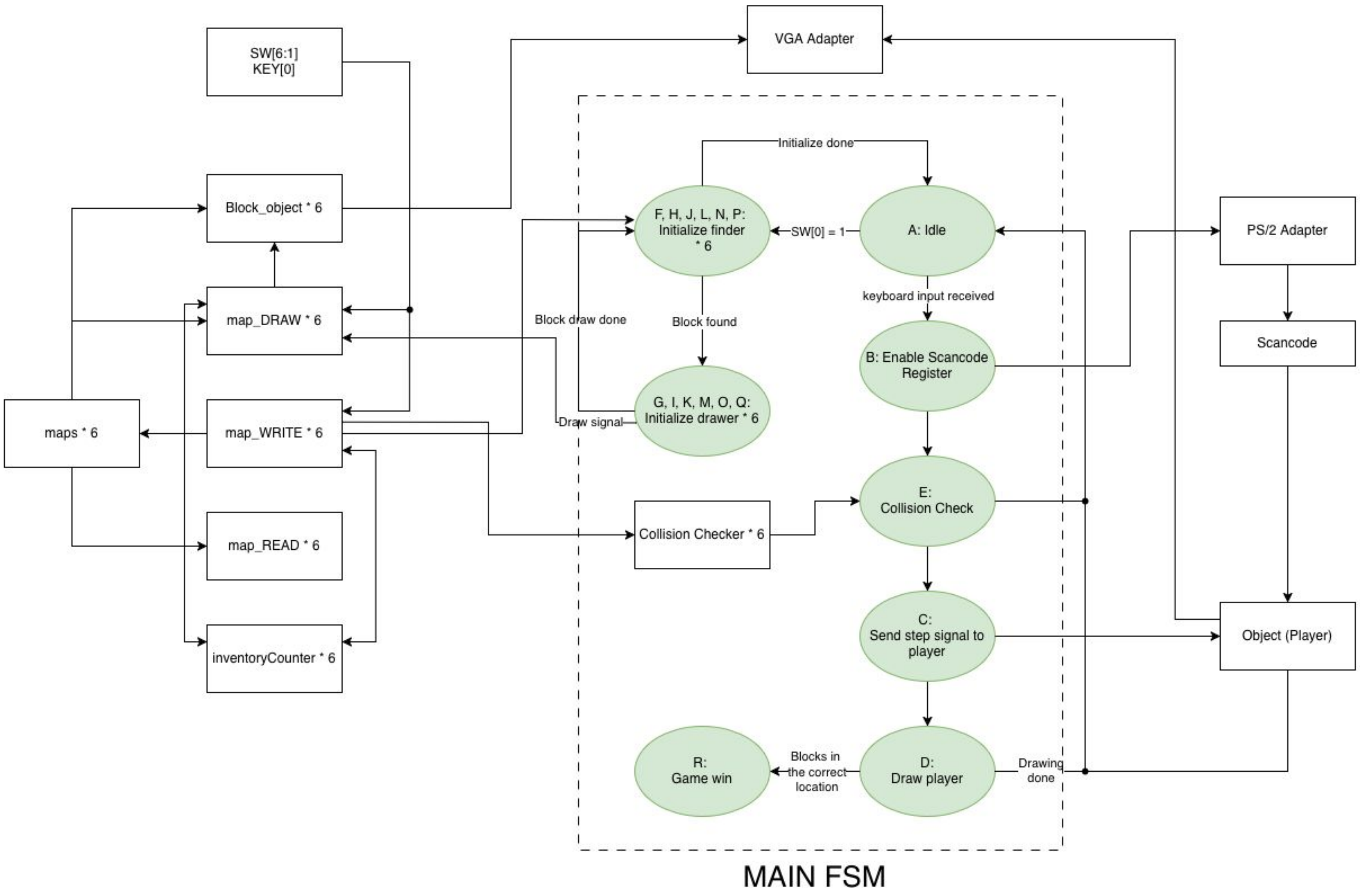
Sub Components Overview

1. Main FSM (Game Logic)
2. VGA Display*
3. PS/2 Keyboard*
4. Block Image Drawing
5. Collision Detection
6. Map storage (Write module)
7. Map initializer
8. Map reader
9. Inventory
10. Goal logic

*(from ECE241 teaching team)

Main FSM (Game Logic)

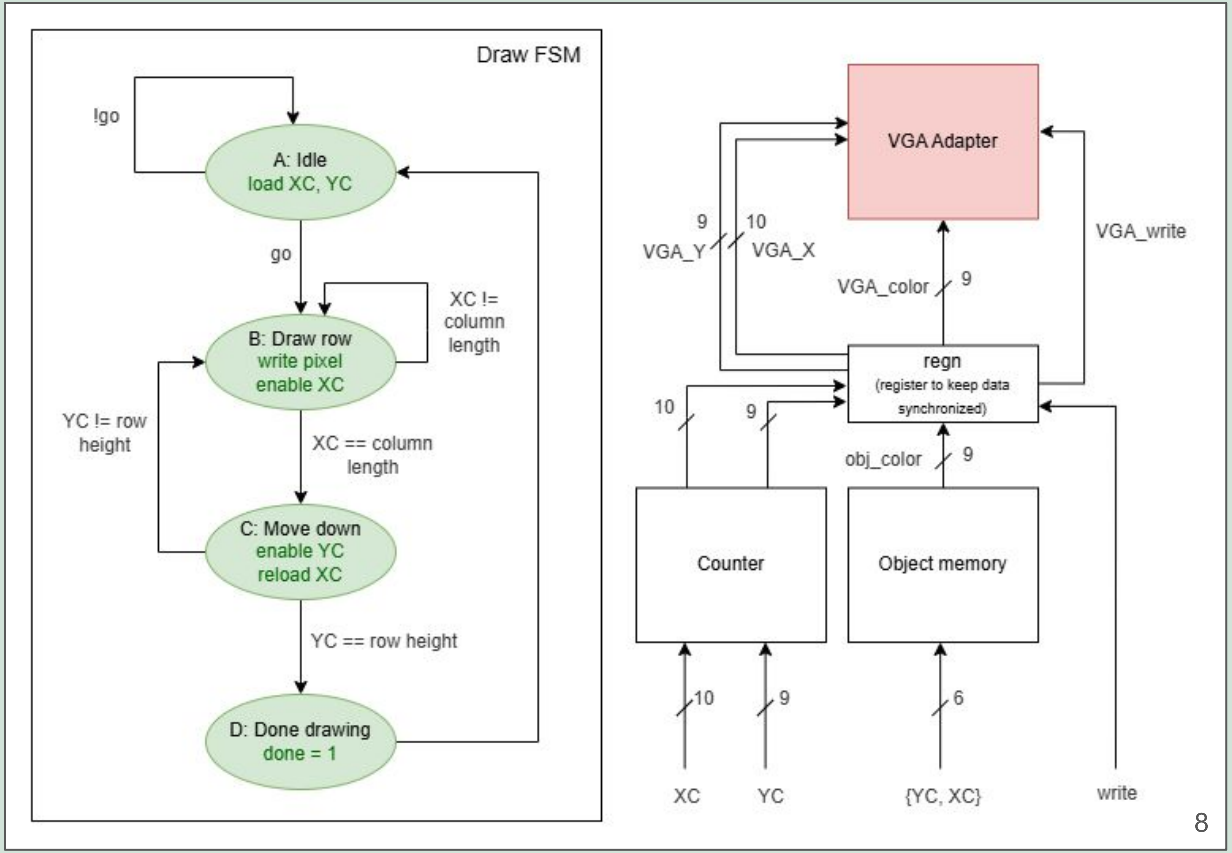
- Main FSM to control the PS/2 input and map initialization process
- For each block type, we have a:
    - Block_object: Used to draw the blocks physically on the VGA
    - map_DRAW: Used to initialize the map at the start
    - map_WRITE: Used to write blocks into the memory
    - map_READ: Used to read blocks from the memory
    - inventoryCounter: Used to count the specific block type
- MUX to control the VGA drawing



SW[6:1]
KEY[0]
VGA Adapter
Block_object * 6
map_DRAW * 6
maps * 6
map_WRITE * 6
map_READ * 6
inventoryCounter * 6
Initialize done
F, H, J, L, N, P: Initialize finder * 6
SW[0] = 1
A: Idle
keyboard input received
Block draw done
Block found
B: Enable Scancode Register
G, I, K, M, O, Q: Initialize drawer * 6
Draw signal
Collision Checker * 6
E: Collision Check
C: Send step signal to player
R: Game win
Blocks in the correct location
D: Draw player
Drawing done
PS/2 Adapter
Scancode
Object (Player)
MAIN FSM

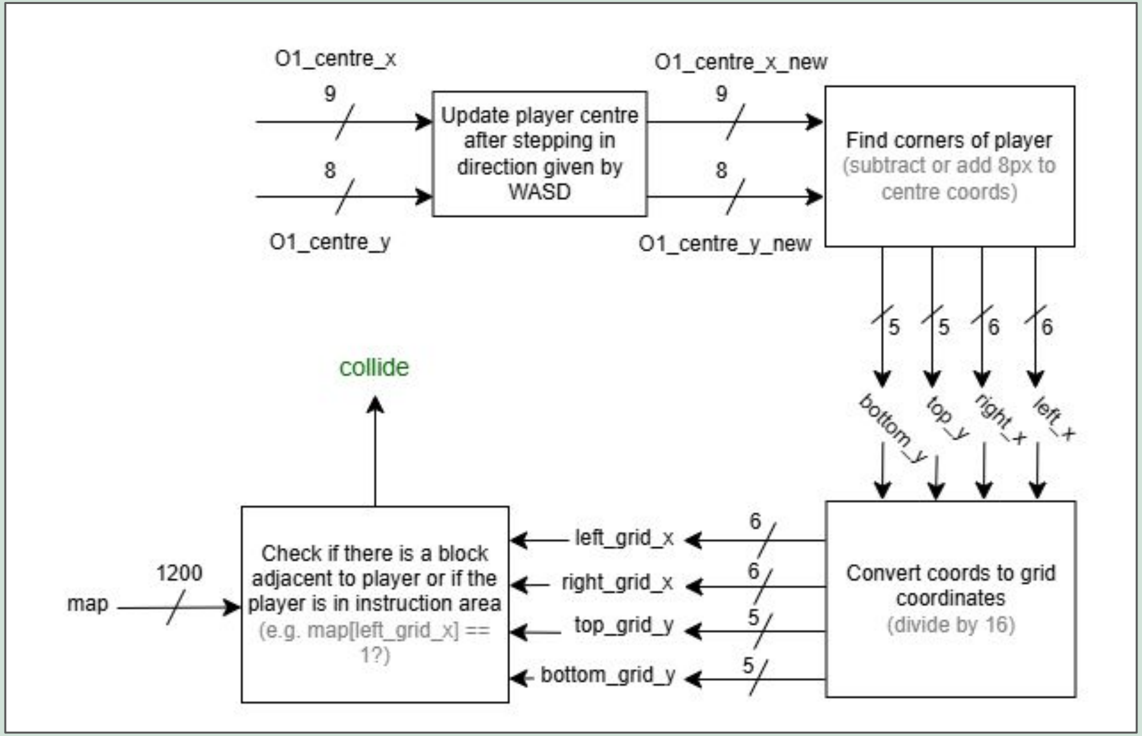
Block Drawing

- Used to draw individual object blocks, modified from the *"object"* demo given by ECE241 teaching team
- Each object block is a `.mif` file stored in memory



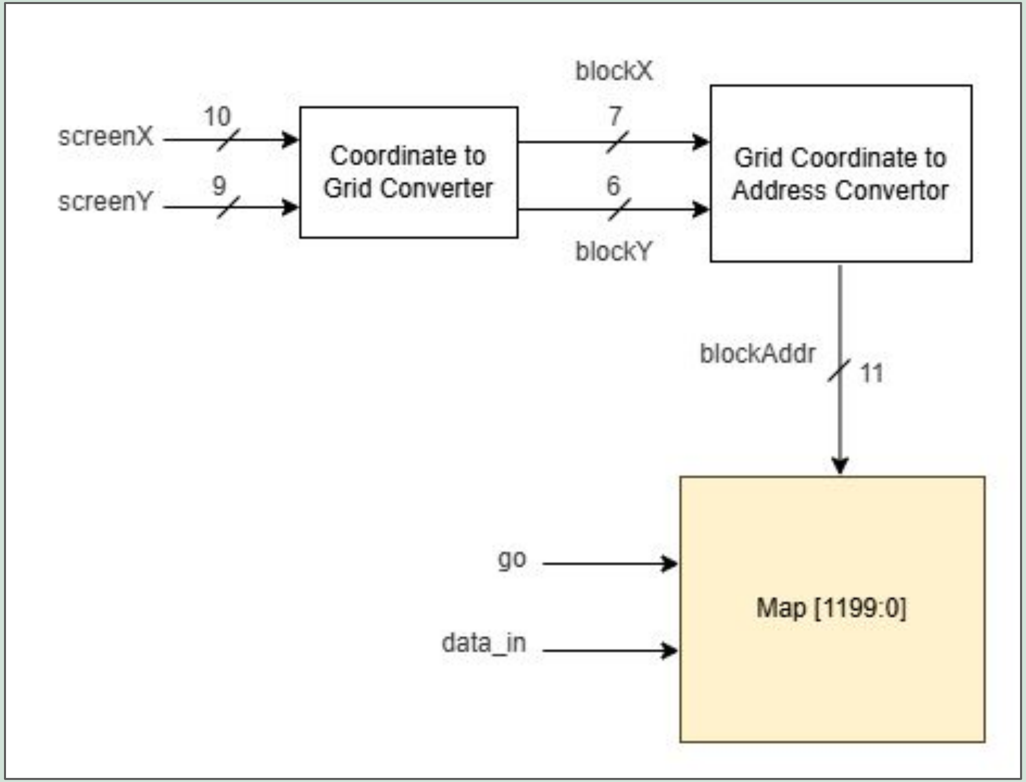
Collision Detection

- Detects whether player will hit an object block or the instruction section of the map
- Used so that blocks don't get erased by the player



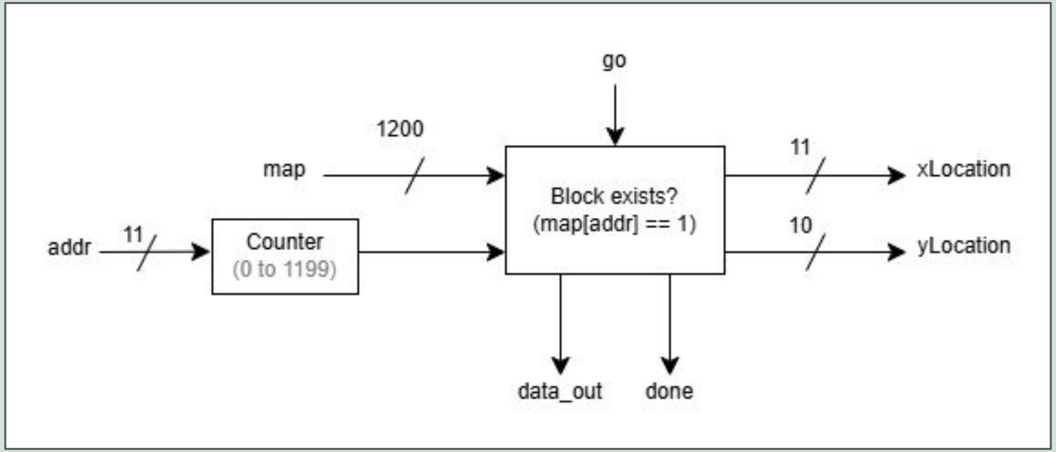
Map Storage (Writer)

- Stores and updates the object map
- Instantiated 6 times — once per block type
- Updates whenever `canPlace` or `canBreak` is true and a block is placed or broken



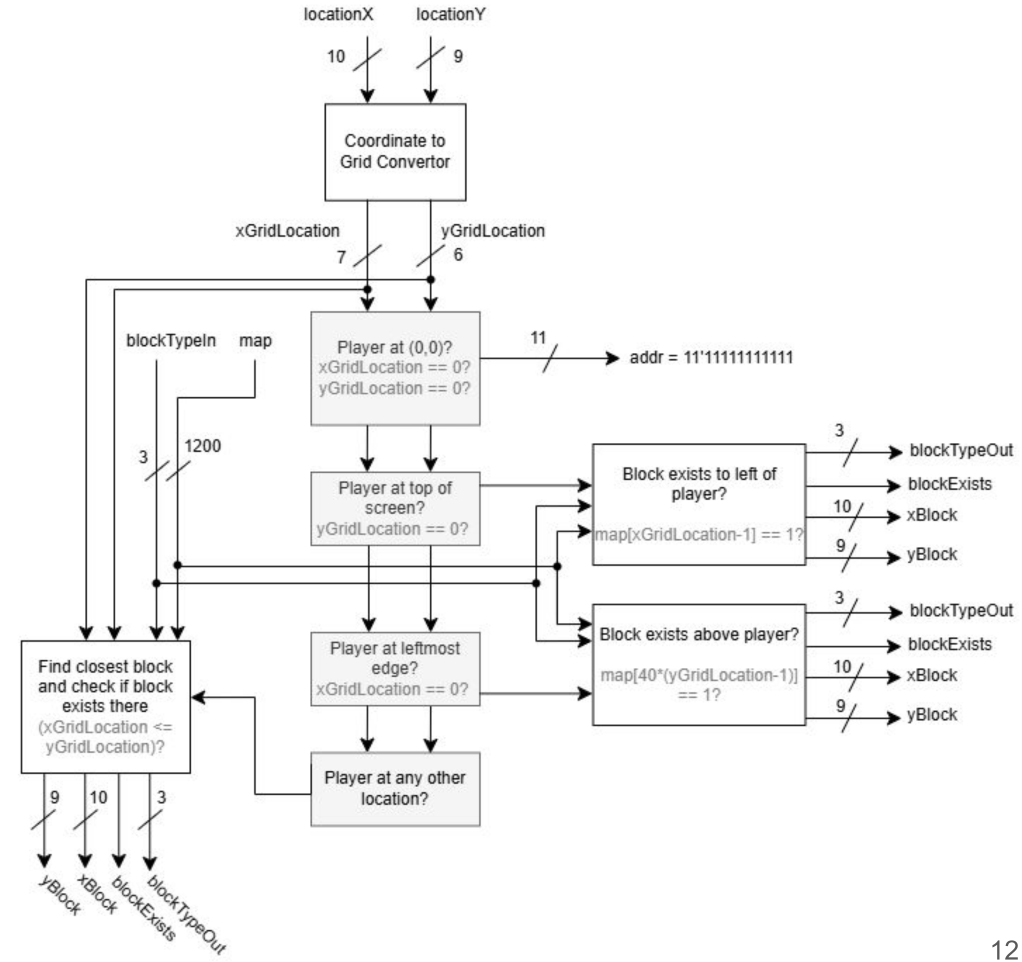
Map Initializer

- Reads entire map storage and outputs an x and y coordinate if a block exists at that location
- Used to draw the map at the start of the game



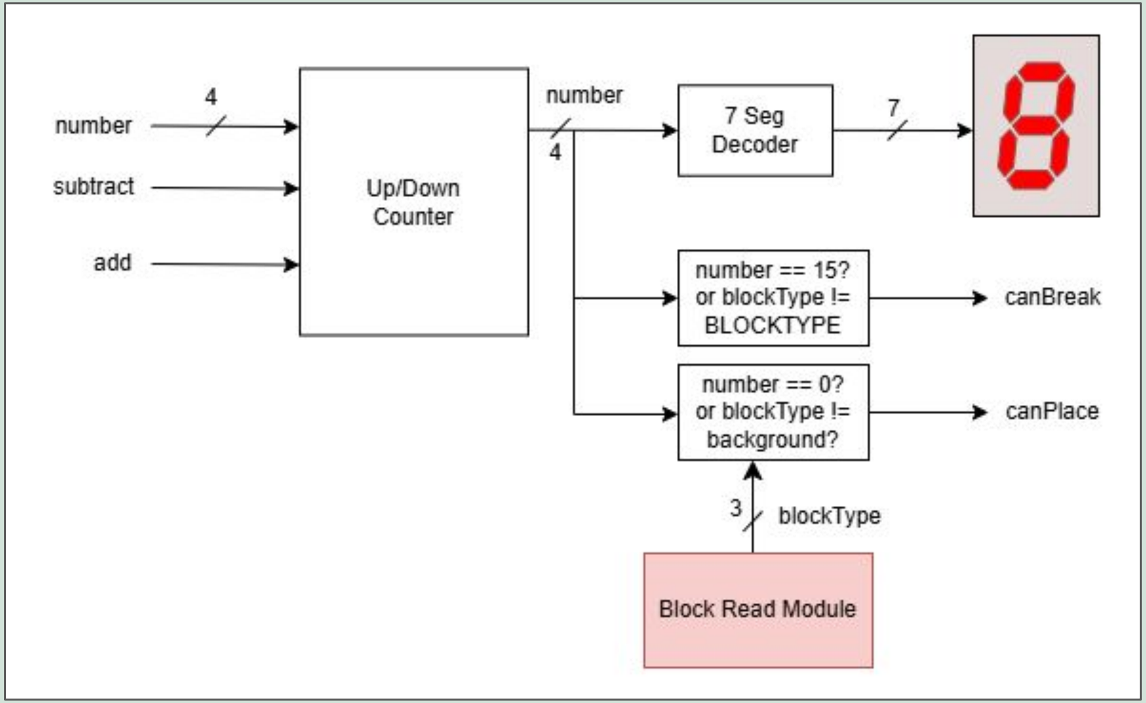
Map Reader

- Takes in location of player and outputs blockType of a block within 1 block to the upper left of the player
- Blocks to left of player take priority, blocks above player take second priority
- This module is used for placing and breaking logic



Inventory

- Keep track of how many blocks player has
- Used for placing (must have blocks in inventory to be able to place)
- Breaking adds 1 to the correct inventory
- Displayed on HEX_{5-0}



Goal Logic

- Takes the 6 map arrays as input
- Reads if the desired blocks are at a specific location
- Win_con goes high if blocks exist at the right location.

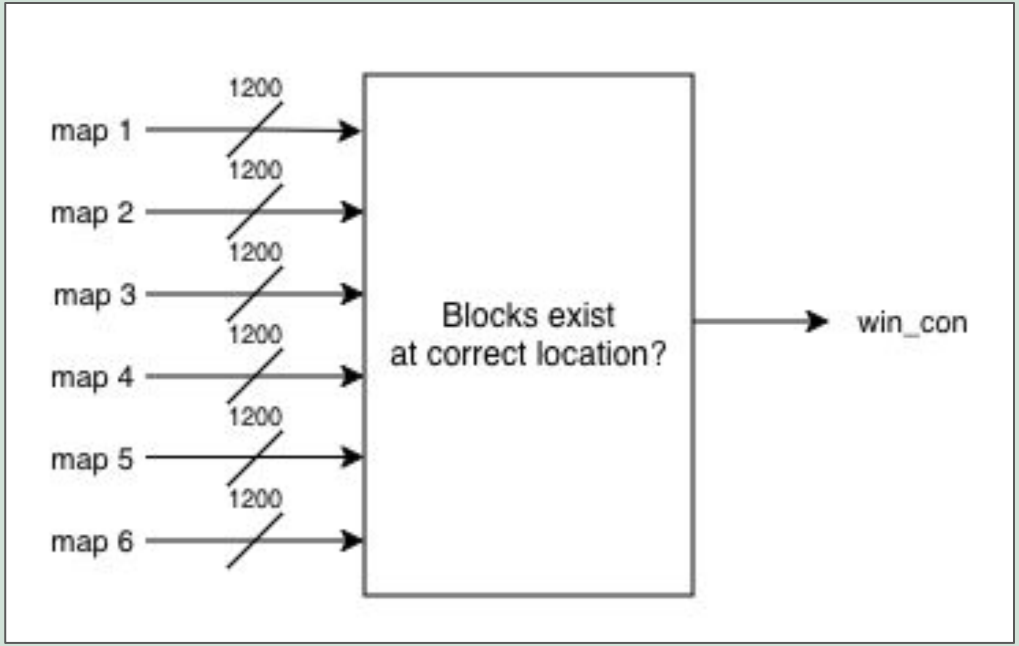
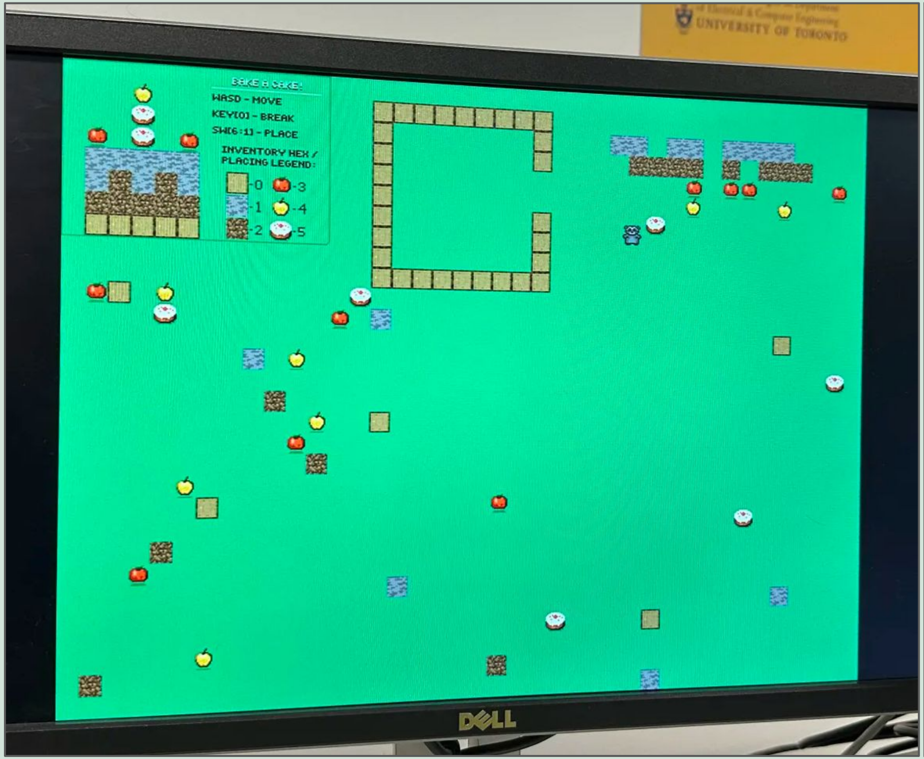
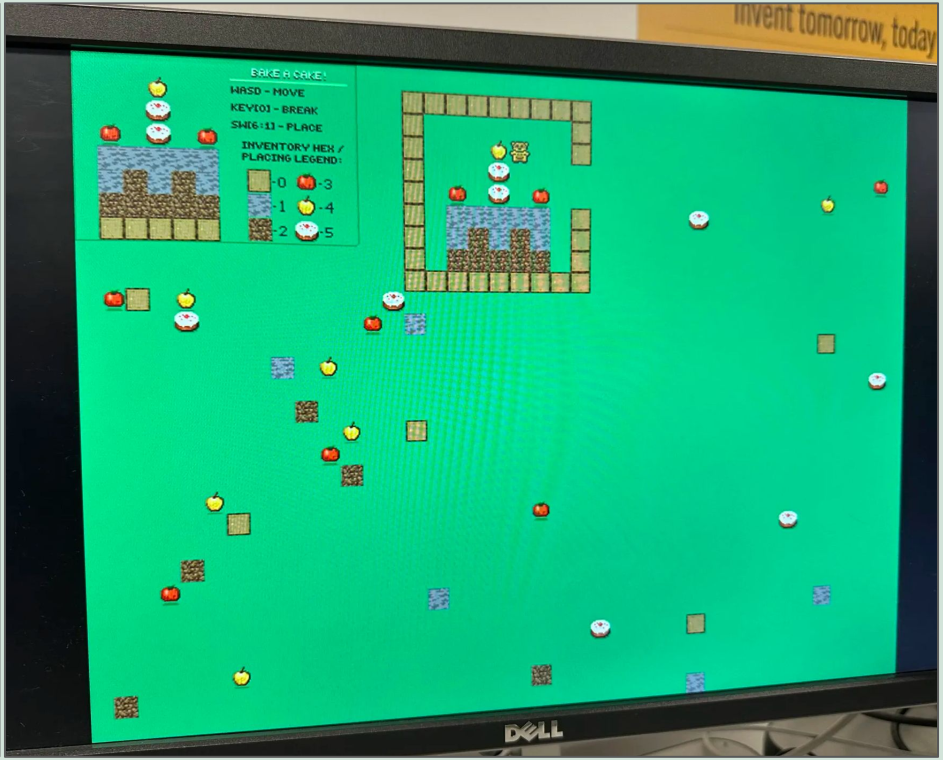


Photo Gallery





Bugs and Issues Encountered

- Not erasing blocks when player moved over them
  - collision detection
- No 2D regs to store maps
  - used 1D reg and converted the addresses to locations
- Blocks placed on top of eachother
  - use canPlace from inventory to detect if can place
- Blocks not initializing properly (Skipping blocks)
  - Use a check signal to check if fsm has done processing

Future Improvements

- Add inventory on screen (like a hotbar)
- Make food items consumable and have a hunger bar
- More “levels”/things to build
- Sound when placing and breaking blocks, ambient noise in general
- Mobs?

Questions?